

Más capas en el perceptrón multicapa (Iris)

Introducción a la Inteligencia Artificial

Zeus Sarmiento

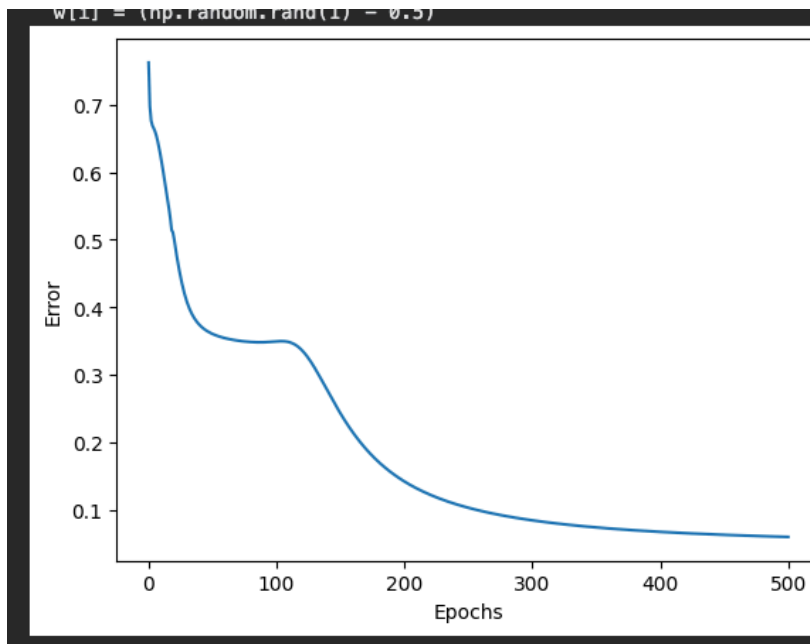
11/09/2026

Enlaces de Colab:

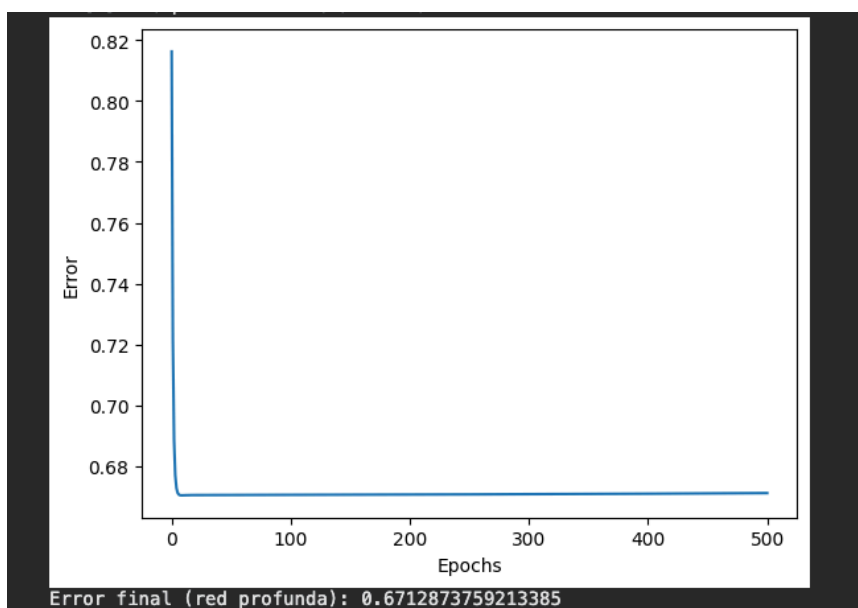
- https://colab.research.google.com/drive/1hJ_DqFAwZad6uB4KAahE5VZZu6EN8JjR?usp=sharing
- <https://colab.research.google.com/drive/1UPAOXSmilgRxXx3UF8zzR0JTnxjcmow-?usp=sharing>

Capturas: curvas de error/pérdida

Notebook 04



Error final (original): 0.05979299596824084



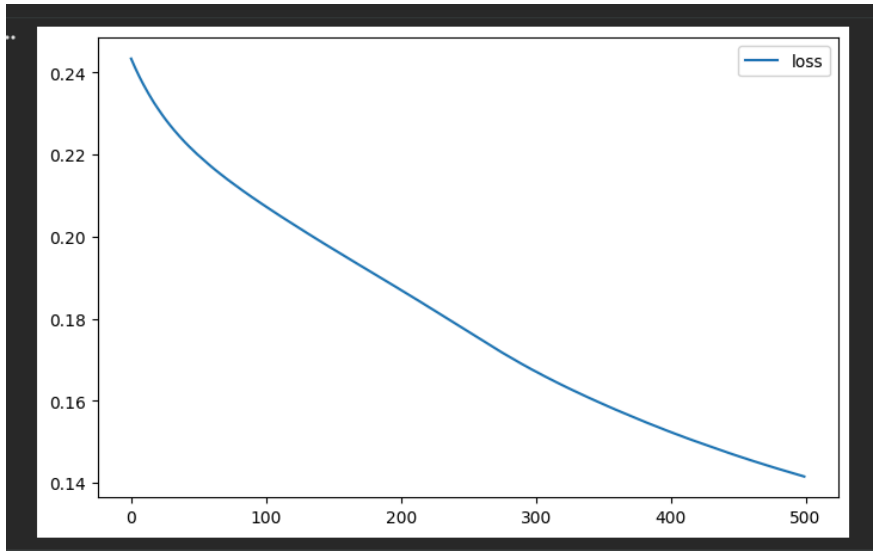
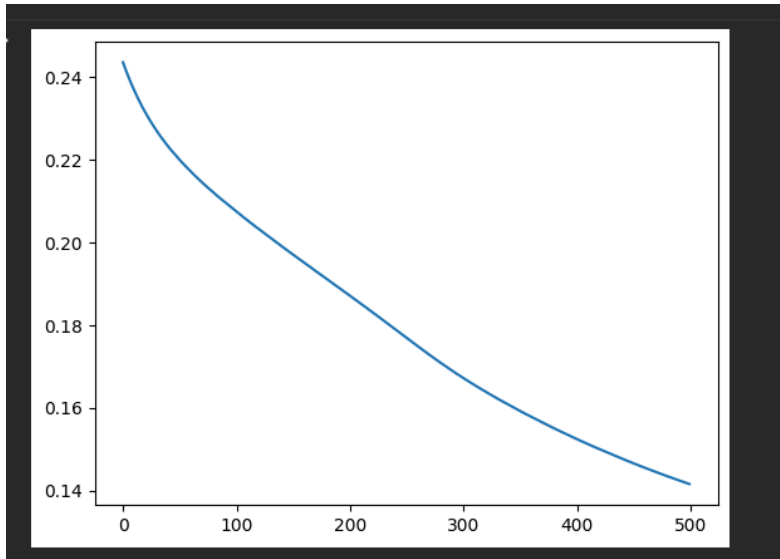
Notebook 05

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Total params: 27 (108.00 B)
 Trainable params: 27 (108.00 B)
 Non-trainable params: 0 (0.00 B)

```

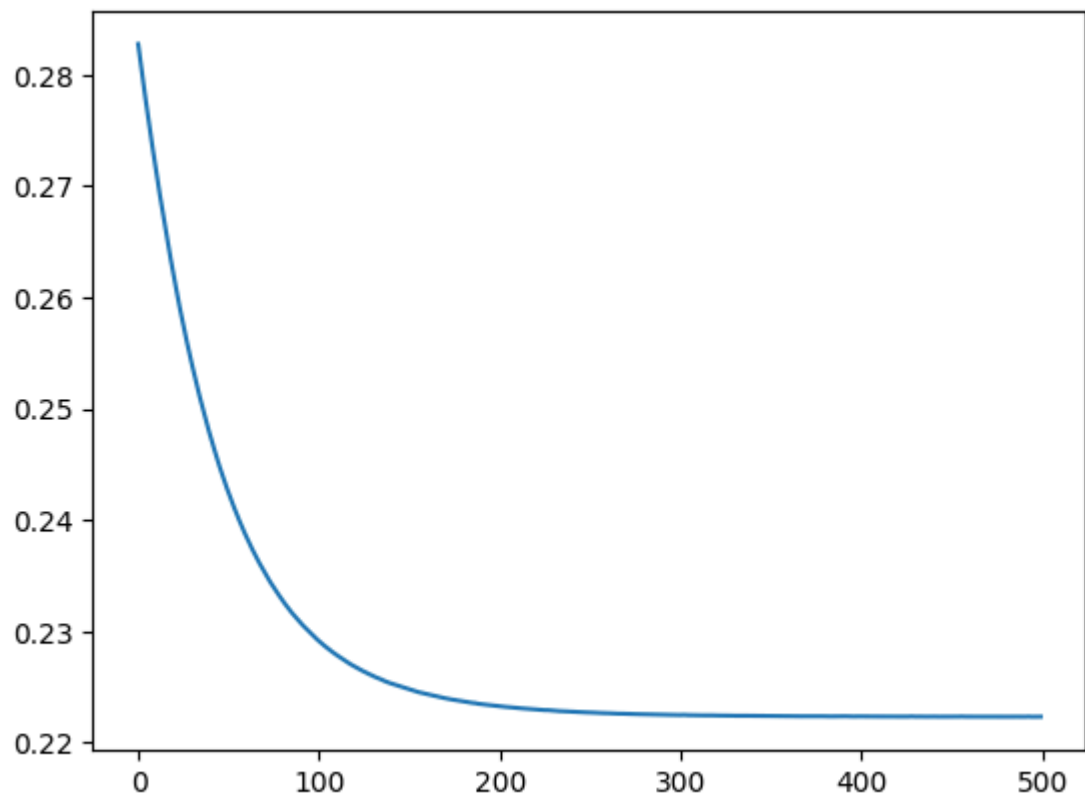
Epoch 497/500
5/5 ██████████ 0s 7ms/step - loss: 0.1419
Epoch 498/500
5/5 ██████████ 0s 8ms/step - loss: 0.1418
Epoch 499/500
5/5 ██████████ 0s 7ms/step - loss: 0.1417
Epoch 500/500
5/5 ██████████ 0s 8ms/step - loss: 0.1416
    
```



Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)

```
Epoch 497/500
5/5 ————— 0s 7ms/step - loss: 0.2223
Epoch 498/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 499/500
5/5 ————— 0s 7ms/step - loss: 0.2223
Epoch 500/500
5/5 ————— 0s 7ms/step - loss: 0.2223
```



Loss final (red profunda): 0.22230865061283112

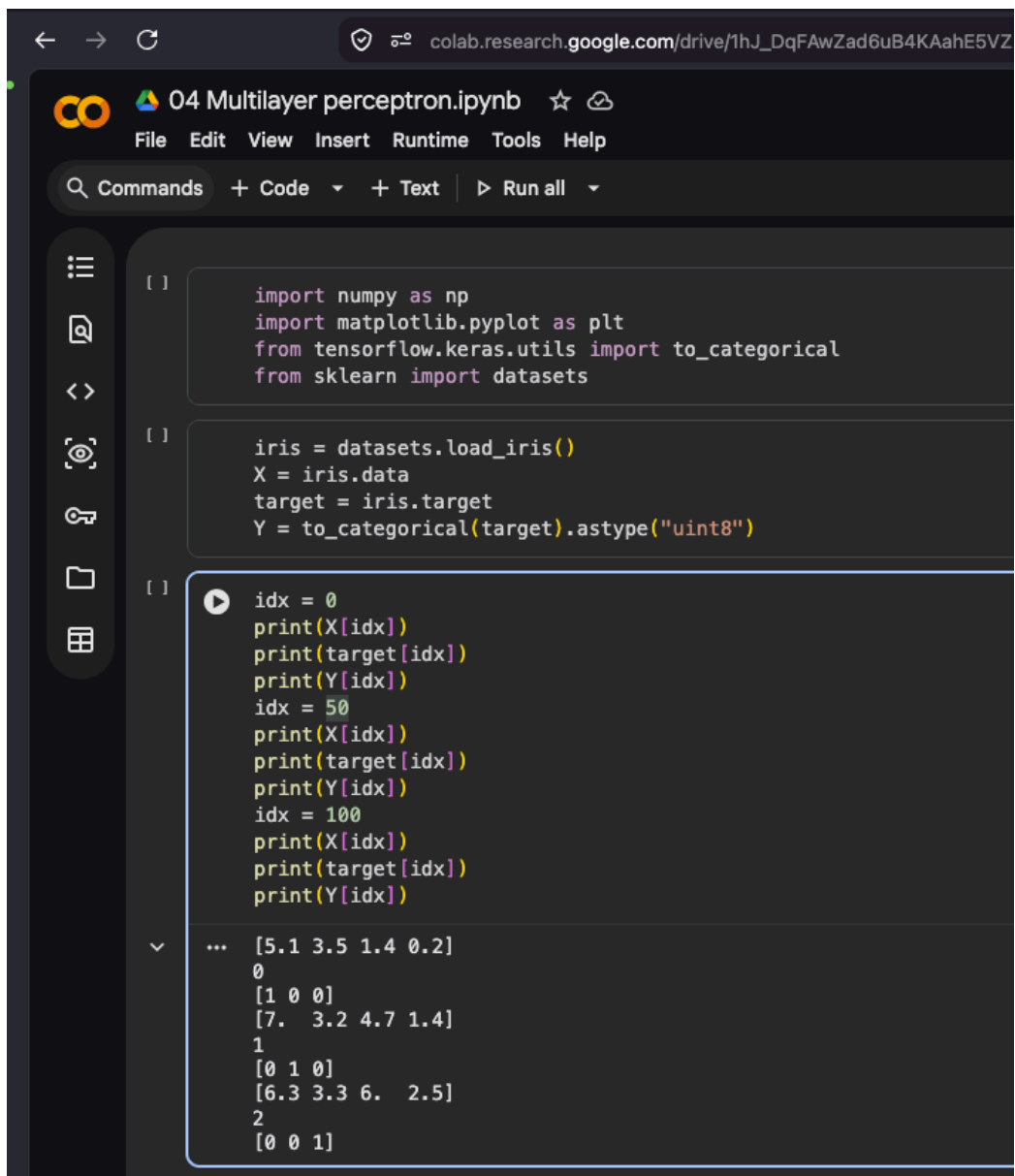
Análisis de resultados

En NumPy la red de $4 \times 3 \times 3 \times 3$ sí bajó el error: terminó en 0.0597. Cuando le puse las dos capas extra se fue a 0.671. Casi no se movió después de las primeras épocas; se estancó y de hecho empeoró. En Keras pasó algo parecido. La original cerró cerca de 0.14 y la profunda en 0.222, y desde como la época 200 la curva ya no bajaba. O sea, en las dos implementaciones añadir capas no ayudó, al contrario. Como no toqué ni la η (0.03), ni las 500 épocas, ni la sigmoide, ni el MSE, lo que cambió fue nada más la profundidad.

Las curvas de la notebook de NumPy y las de Keras no se parecen mucho, aunque la topología sea la misma. Las dos bajan y luego se aplanan, pero los números y la forma no coinciden, y tiene sentido. En NumPy el error es el promedio de la suma de los tres cuadrados de cada ejemplo; Keras con MSE también promedia sobre las tres salidas, entonces el loss sale más chico. Además NumPy va ejemplo por ejemplo y recorre Iris en el orden en que viene (primero setosa, luego versicolor, luego virginica), y Keras usa `fit` con batch de 32 y revuelve los datos. Los pesos tampoco se inicializan igual: en NumPy es `rand()` - 0.5 y en Keras es Glorot. Y uno está hecho a mano con ciclos en Python y el otro va vectorizado. Por eso, aunque en el pizarrón sea la misma red, al correrlas no dan la misma gráfica.

Sí me hace sentido que la red más profunda no haya aprendido mejor. Iris es un problema chico, 150 ejemplos y 4 atributos, y las clases ya se separan bastante; no ocupa tantas capas. Encima todas las activaciones son sigmoides, y la derivada de la sigmoide a lo mucho es 0.25, entonces al apilar cuatro los δ de las primeras capas se hacen muy chicos (el gradiente se desvanece) y casi no se actualizan los pesos. El MSE tampoco es la pérdida más natural para un one-hot de 3 clases. Eso se ve en las gráficas: la profunda de NumPy se aplanan en 0.67 y la de Keras en 0.22, las dos peor que su versión con una sola capa oculta. En este caso, más capas no significó menos error.

Evidencias de ejecución



The screenshot shows a Google Colab notebook interface. The browser address bar at the top displays the URL: `colab.research.google.com/drive/1hJ_DqFAwZad6uB4KAahE5VZ`. The notebook title is "04 Multilayer perceptron.ipynb". The menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a toolbar with "Commands", "+ Code", "+ Text", and "Run all". A left sidebar contains icons for file management and viewing. The notebook contains three code cells:

```
[ ] import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical
from sklearn import datasets
```

```
[ ] iris = datasets.load_iris()
X = iris.data
target = iris.target
Y = to_categorical(target).astype("uint8")
```

```
[ ] idx = 0
print(X[idx])
print(target[idx])
print(Y[idx])
idx = 50
print(X[idx])
print(target[idx])
print(Y[idx])
idx = 100
print(X[idx])
print(target[idx])
print(Y[idx])
```

The output of the third cell is displayed below the code:

```
... [5.1 3.5 1.4 0.2]
0
[1 0 0]
[7.  3.2 4.7 1.4]
1
[0 1 0]
[6.3 3.3 6.  2.5]
2
[0 0 1]
```

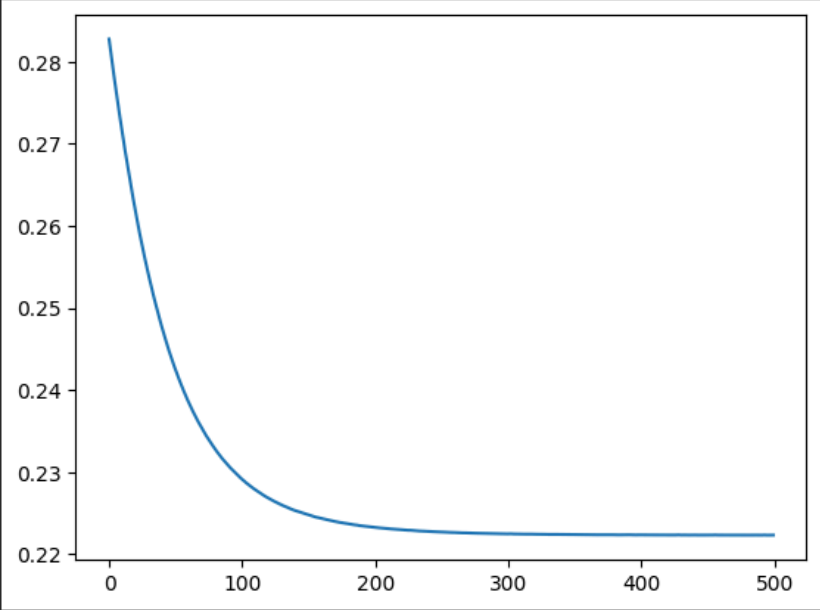
colab.research.google.com/drive/1UPAOXSmilgRxXx3UF8zzR0JTnxjcm

05 Keras - multilayer perceptron - iris.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[13] plt.plot(history.history['loss'])
      plt.show()
      print("Loss final (red profunda):", history.history['loss'][-1])
```



Loss final (red profunda): 0.22230865061283112

```
[14] model.predict(np.array([[3, 3, 1, 1]]))
```

1/1 0s 71ms/step
array([[0.33448166, 0.3354166 , 0.33735344]], dtype=float32)

[] Start coding or generate with AI.